

IOCP - 2

MM4220 게임서버 프로그래밍
정내훈

지금 까지

- 싱글스레드 IOCP 서버 제작
- 멀티스레드 프로그래밍 학습
- 멀티쓰레드 IOCP

IOCP

● 시작

- GetQueuedCompletionStatus를 별도의 쓰레드로 옮기기
- 해보자!
- worker thread 작성
 - 개수? : 코어의 개수
 - `thread::hardware_concurrency()`
- Main함수의 GQCS루프를 worker thread로 이전

멀티쓰레드 IOCP

- 문제점

- Thread는 별도의 함수이므로 많은 변수를 전역변수로 이전해야 한다.

```
array < SESSION, MAX_USER> g_users;  
// 또는 unordered_map<int, SESSION> g_users;  
SOCKET g_s_socket;  
HANDLE g_h_iocp;
```

기존 싱글쓰레드 컨테이너

멀티쓰레드 IOCP

- 멀티쓰레드 프로그램의 가장 큰 문제

Data Race

- 두번째 문제
 - `std::mutex` 사용으로 인한 성능 저하

멀티쓰레드 IOCP

● Data Race? 잘 돌아가는데?

- 지금 서버가 부하가 너무 적어서 두개 이상의 쓰레드가 동시에 실행될 경우가 없다.
- 쓰레드가 하나만 실행되는 경우 Data Race 발생 확률이 매우 낮다.
- 하지만 실제 서버에서는 **많은 클라이언트**가 접속해 있어 처리해야 할 작업이 많기 때문에 Data Race가 발생하고 서버가 사망한다. => Data Race를 없애야 한다. => 다음 챕터에서 많은 클라이언트로 실습을 해보자.

멀티쓰레드 IOCP

- 여러 쓰레드에서 g_users 자료구조에 접근
 - Broadcasting
 - 검색
 - 다른 유저에게 나의 정보를 보내야 함.
 - 접속 시, 이동 시, 접속종료 시
 - index 를 사용한 session 참조
- 문제 : g_users 에서 DATA RACE, mutex 는 느려서 사용금지

DATA RACE : std::mutex

- SESSION 삭제/추가는 자주 발생하지 않기 때문에 mutex 사용 시 큰 성능저하는 없어 보임
- ID 를 사용한 SESSION 검색이 문제
 - mutex를 사용해야 한다.
- 검색은 자주 발생하므로 성능에 큰 영향을 미친다. => 서버를 멀티쓰레드로 구현한 의미가 없다 => **mutex 사용 금지**

DATA RACE : array

```
array <SESSION, MAX_USER> g_users;
```

- 검색 비용 **제로**
- 컨테이너 자체는 Data Race가 없음
 - 컨테이너의 크기가 변하지 않고, 원소들의 메모리 주소가 변하지 않음. (vector와 map이 곤란한 이유)
- 빈 공간 문제? NO PROBLEM!
 - 최대 동접에서 잘 되면 모든 것이 OK
- SESSION 의 **재사용** 문제가 발생, id **재사용** 문제도 같이 발생
 - Id 재사용 문제 : 주민등록번호 재사용 문제와 같다.
 - 전과 기록 계승!!!

DATA RACE

```
unordered_map <int, SESSION> g_users;
```

- 엄청난 Data Race
 - 많은 곳에서 g_users를 검색함
 - SESSION 추가/삭제 시 **g_users** 컨테이너 자체가 수정되므로 Data Race 발생
- 하지만 다행 : SESSION 객체들의 메모리 주소는 바뀌지 않음.
 - std::vector와는 다르다.

다중 접속 서버

- `unordered_map<int, SESSION>` Data Race 대책
 - PLAN_A : 모든 접근에 `std::mutex` 사용
 - PLAN_B : `std::array` 사용
 - PLAN_C : `concurrent_unordered_map` 사용

DATA RACE : concurrent_map

- PPL (Parallel Patter Library)

- Visual Studio에서 제공하는 **고성능** 멀티쓰레드 Safe 자료 구조
- Vector, Set, Map이 있음.
- Intel TBB에 좀더 많은 자료구조가 있음.

- Concurrent_unordered_map

```
concurrent_unordered_map <int, SESSION> g_users;
```

- mutex를 사용하지 않아도 SESSION 추가/검색 시 Data Race가 발생하지 않는다.
- 단점 :
 - **Erase가 안된다.**
 - **해결 방법 #1** : SESSION 를 *SESSION 으로 변경
 - 컨테이너의 size가 줄어들지 않는 사소한 메모리 낭비는 무시...
 - **해결 방법 #2** : 삭제도 되는 자료구조 사용
 - Intel TBB concurrent_hash_map, 또는 자체 제작 (Lock Free SkipList)

다중 접속 서버

- `unordered_map<int, SESSION>` Data Race 대책
 - PLAN_A : 모든 접근에 `std::mutex` 사용
 - PLAN_B : `std::array` 사용
 - PLAN_C : `concurrent_unordered_map` 사용
- PLAN_B, PLAN_C에서 사용중인 Session에 대한 `destruct` 대책?
 - PLAN_A : Never Destruct
 - PLAN_B : `std::atomic<shared_ptr<SESSION>>`
 - PLAN_C : advanced multithread memory reuse technique. (예: hazard pointer, EBR)

DATA RACE : array

```
array <*SESSION, MAX_USER> g_users;
```

● Id 재사용 문제

- 접속 시 새 아이디부여

- 정기점검 1 주일 동안 : 최대 동접 5000 => DAU 2,5000 => 일간 접속 25,0000 => 일주일 접속 210,0000

- 아이디 재사용

- 같은 캐릭터는 같은 SESSION 재활용
 - SOCKET 이 변경되는데 문제는 없는가?
- 캐릭터와 ID 매핑 테이블 필요.
 - Add/Read Only, 제거/수정 없음

Destruct?

- 클라이언트는 접속과 logout을 반복한다.
- logout시 SESSION을 destruct하지 않으면 Memory Full로 서버가 사망한다.
 - 추가 요구 사항 : 일정 시간 이상 반응이 없으면 강제로 destruct 해야 한다.
- 운영체제의 socket 객체는 개수에 제한이 있다. close()를 해서 재사용 해야 한다.
- 멀티쓰레드 서버의 경우 destruct 시 다른 쓰레드에서 해당 SESSION을 참조하고 있을 수 있다.
 - 함부로 destruct하면 오류가 발생한다. Close 된 소켓에 대한 send/recv 요청 오류

Never Destruct?

- 불가능
- 이유 : 메모리 부족
 - 메모리가 아무리 많아도 오랜 시간 서버가 살아 있으면 memory full
- 이유 : 운영체제 커널 SOCKET 개수 제한
 - 커널 Configuration으로 늘릴 수 있지만 오랜 시간 서버가 살아있으면 역시 한계

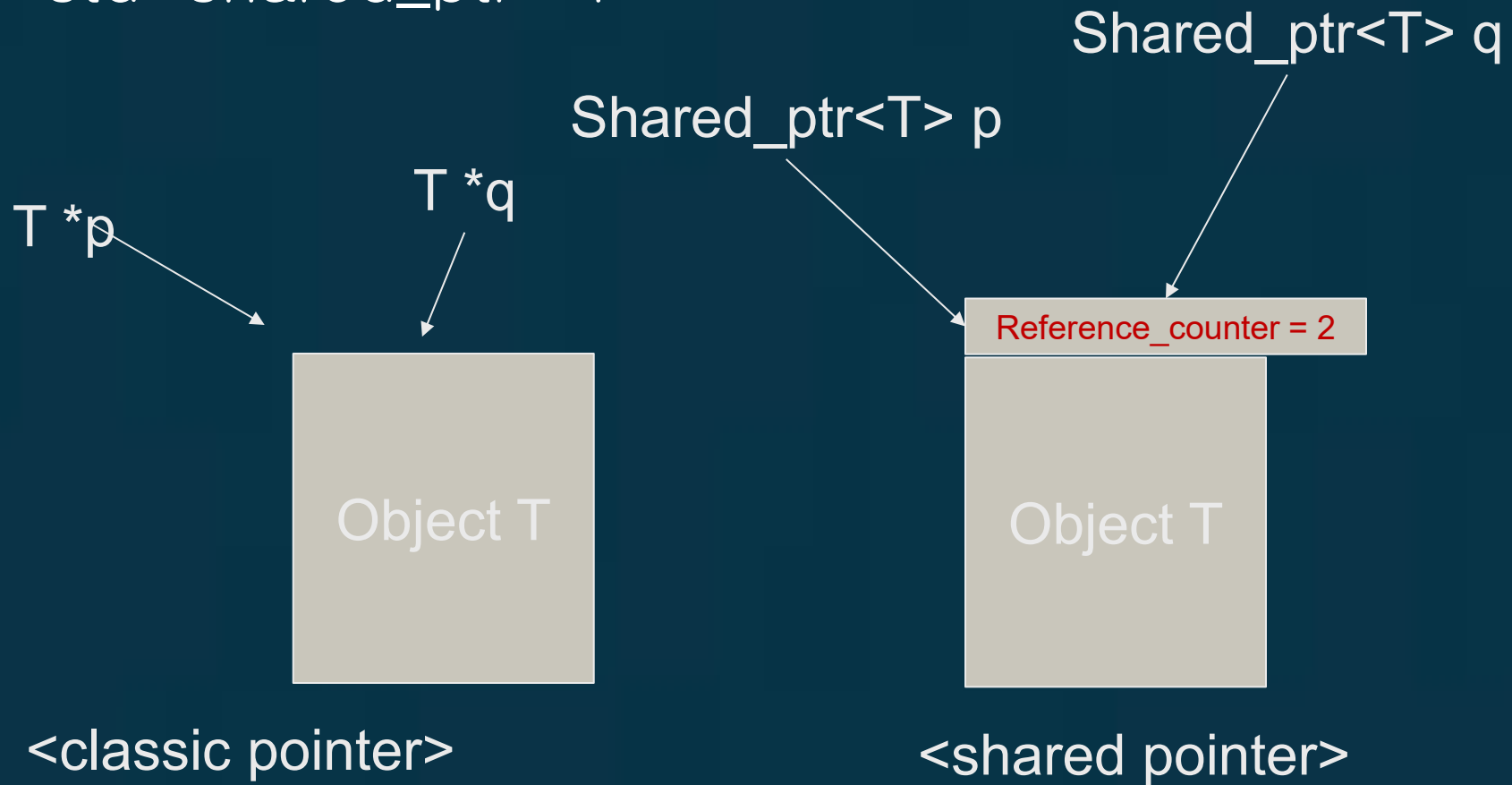
Destruct? : shared_ptr (1/3)

- std::shared_ptr <T>

- C++11 표준에서 도입된 **스마트 포인터**.
- **스마트 포인터**
 - delete 명령어가 없다. (최신 언어들은 다 그러하다!)
 - 앞으로 사용 되지 않음을 자동으로 감지해서 자동으로 delete 한다.
 - 내가 참조하고 있는 SESSION은 절대 destruct 되지 않는다.
 - 따라서 destruct를 걱정하지 않고 SESSION 객체를 사용할 수 있다.
 - 따라서 포인터 사용시 따르는 위험성을 100% 제거해 준다.
 - 위험 1 : delete를 안 해서 메모리 FULL
 - 위험 2 : 사용 중인 객체를 delete
 - 위험 3 : 두 번 delete
- 사소한 단점 : circular pointing 은 해결 불가능
 - 하지만 std::weak_ptr<T>로 해결 가능

Destruct? : shared_ptr (1/3)

- `std::shared_ptr <T>`



Destruct? : shared_ptr (1/3)

- `std::shared_ptr <T>`
- 생성
 - `std::shared_ptr <T> ptr = std::make_shared<T>(생성자 파라미터);`
- 객체 A의 소멸 조건 (다 같은 이야기)
 - 모든 shared pointer 가 A를 가리키지 않을 때
 - A 를 가리키는 shared pointer 가 하나도 없을 때
 - 객체 A 의 reference_counter 가 0 되었을 때

Destruct? : shared_ptr (2/3)

- 치명적인 단점

- shared_ptr는 atomic한 자료구조가 아니다.
 - mutex를 사용하지 않으면 오류가 발생한다,

틀린 프로그램

```
// 전역변수
```

```
shared_ptr <T> a; shared_ptr<T> b;
```

```
a = b;           // Data Race로 오류 발생
```

```
b = nullptr;    // Data Race로 오류 발생
```

Destruct? : shared_ptr (2/3)

- 치명적인 단점
 - shared_ptr는 atomic한 자료구조가 아니다.
 - mutex를 사용하지 않으면 오류가 발생한다,
- 해결책
 - C++20에서는 **atomic<shared_ptr<T>>**를 사용하면 atomic하게 동작한다.
- 사소한 부작용
 - 내부적으로 mutex를 사용한다. 따라서 그냥 shared_ptr<T>보다 느리다.
 - 대책 : 사용 횟수의 최소화
 - 대책 : copy 를 최소화 해야 한다
 - 매개변수 사용시 const reference 형태로 전달하자
 - 지역 변수에서 사용할 때는 atomic 을 사용하지 말자.

Destruct? : shared_ptr (3/3)

- 사용시 주의

- 사용하기 전에 nullptr 인가를 먼저 검사하고, 복사해서 사용해야 한다.
 - 아까는 nullptr가 아니었지만, 지금은 nullptr일 수 있다.
 - 싱글쓰레드 프로그래밍을 할 때와는 다르다.
 - 방법)
 1. `shared_ptr<SESSION> cl = g_clients[client_id];`
 2. `if (nullptr == cl) return; // 이후 cl 이 destruct 될 때 까지 안전`
 3. `cl->m_x += 2;`

- 그래도 사용해야 하는가?

- Mutex 없이 멀티쓰레드 컨테이너에 SESSION을 저장하기 위해서는 필수
또 ID, SESSION 재사용 문제 해결을 위해서
 - Data Race로 인한 오동작 방지. (delete 와 access 가 겹칠 때)
- 성능 비교 (벤치마킹) 필요 => 다음 챕터에서 실습

DATA RACE

- SESSION

- No Data Race

- m_id
 - m_prev_size
 - m_s
 - M_recv_over

- Data Race

- m_name
 - m_x, m_y

```
class SESSION
{
    int            m_id;
    unsigned char  m_prev_size;
    SOCKET         m_s;
    EXP_OVER       m_recv_over;

    char           m_name[MAX_NAME];
    int            m_x, m_y;
};
```

- no data race : 한 순간에 오직 한 스레드에서만 접근 할 때 == 두 개 이상의 스레드에서 동시에 접근하지 않을 때
- no data race : 생성시 (멀티스레드에 참가하기 전) 값이 정해진 후 다른 아무도 값을 변경하지 않을 때

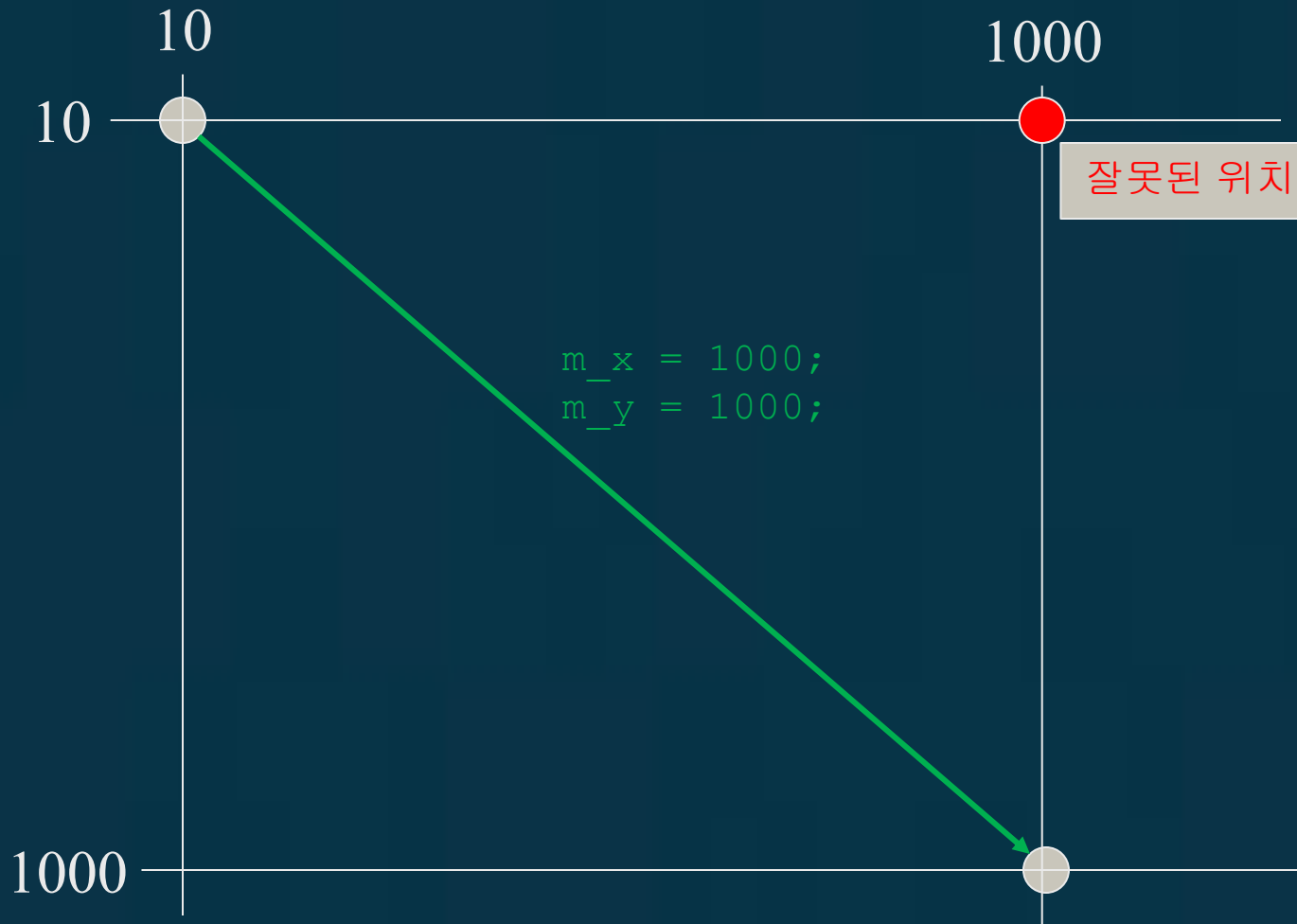
DATA RACE

- 정책

- mutex로 보호한다.
- atomic 자료구조를 사용한다.
 - `bool => std::atomic<bool>`
 - `float => volatile float; // atomic은 아니지만 오류는 줄어든다.`
- 약간의 부정확함을 감수한다.
 - 성능도 향상된다.
 - 세심한 주의가 필요.
 - 예) 플레이어가 이동 가능 할 때
 - `m_x, m_y`는 보호하지 않는다. (성능을 위해 오류를 감수한다.)

DATA RACE

- Atomic 하지 않은 이동 오류 : $(10, 10) \rightarrow (1000, 1000)$



DATA RACE

```
unordered_map <int, SESSION> clients;
```

- 엄청난 Data Race
 - 거의 모든 함수에서 clients를 건드림
 - **Clients 컨테이너 자체가 수정**되므로 Data Race대량 발생
 - 원소들의 메모리 주소는 바뀌지 않음. 검색 자료구조에서 Data Race발생
 - 원소 내부에 접근할 때도 여전히 Data Race존재 => 원소마다 mutex 필요
- 해결 방법 #1

```
concurrent_unordered_map <int, SESSION> clients;
```

- **Erase가 안된다** => SESSION 를 *SESSION 으로 변경 필요
- 다른 스레드에서 사용중인 SESSION를 delete하면???
- 해결 방법 #2

```
concurrent_unordered_map <int,
                        atomic<shared_ptr <SESSION>>> clients;
```

- Container의 크기 한계 문제 => 주기적으로 서버 재시작.
- **그럴 바에는 차라리 array 를 쓰자.** => ID 재사용 문제 발생.

DATA RACE

- SESSION

- m_id : no data race
- m_recv_over : no data race
- m_prev_recv : no data_race
- m_s : no data_race
- m_name : data race
- m_x, m_y : data race

```
class SESSION
{
    int            m_id;
    EXP_OVER      m_recv_over;
    unsigned char m_prev_recv;
    SOCKET         m_s;

    char           m_name[MAX_NAME];
    short          m_x, m_y;
};
```

- no data race : 한 순간에 오직 한 스레드에서만 접근 할 때 == 두 개 이상의 스레드에서 동시에 접근하지 않을 때
- no data race : 값이 정해진 후 다른 아무도 값을 변경하지 않을 때

DATA RACE

● m_x, m_y

- PLAN_A : mutex로 보호한다.
 - 단점 : 성능
- PLAN_B : atomic 자료구조를 사용한다.
 - 주의 : mutex를 사용해서 구현하면 PLAN_A와 같다.
 - Non-blocking 자료구조 필요
- PLAN_C : 약간의 오동작을 감수 한다,
 - m_x, m_y는 보호하지 않는다. (성능을 위해 오류를 감수한다.)

DATA RACE

● m_name

– 문제점

- Accept를 해서 SESSION은 생성되었으나 cs_login packet이 도착하지 않았을 때
- Broadcasting 할 때 잘못된 이름 전송

– 해결책

- m_name valid 한 지를 나타내는 변수 **m_state**가 필요하다.
 - SESSION의 존재와 Game World 참가를 구분
- m_state의 의미
 - ST_ALLOC : ACCEPT됨, 아직 cs_login 패킷을 받지 못함,
 - ST_INGAME : 게임 참여 중
 - ST_CLOSE : 삭제를 기다리는 중

DATA RACE

- 실습 : 지난 주 제작한 IOCP 서버를 수정

중간고사

- 4월 24일 목요일
- 시험범위 : 진도 나간데 까지.

정리

- 멀티쓰레드 IOCP MMOG 서버의 뼈대 완성
- 완벽하지 않음
 - 멀티 쓰레드 오류
 - x, y 좌표
 - `atomic<shared_ptr>` 로 인한 성능 저하
 - `concurrent_unorder_map` 으로 인한 작은 memory leak

정리

- 이후의 내용
 - 성능 측정
 - 컨텐츠 구현
 - Quest, Item, Skill, 전투....
 - DB에 연동되는 것 말고 별다른 것 없으므로 PASS
 - 시야
 - 성능, 맵핵 방지
 - 충돌 처리
 - 해킹 방지
 - AI
 - 몇 십만 마리의 몬스터..., 스크립트 연동